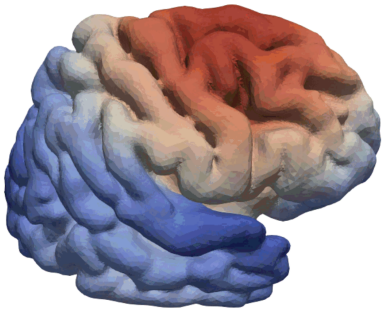
Fisher-Kolmogorov Equation

Modeling Protein Diffusion in Neurodegenerative Diseases

Alberto Taddei

Implementation of finite element methods
Applied Mathematics, Sorbonne Université

March 2026



Contents

1 Introduction **3**

2 Prion Disease Model **4**
2.1 Kinetic Model of Prion Spreading . . . 4
2.2 From the Balance Equation to the Fisher-Kolmogorov Equation . . . 4

3 Computational Model **6**
3.1 Weak Formulation . . . 6
3.2 Residual Definition . . . 6
3.3 Spatial Discretization via Finite Elements . . . 6
3.4 Time Discretization theta method . . . 6
3.5 Residual vector and Jacobian matrix . . . 8
3.6 Implementation Details and Software Stack . . . 9
3.7 Implementation of the Diffusion Tensor . . . 9

4 Well–posedness and Stability **11**
4.1 Standing assumptions. . . . 11
4.2 Why Lax–Milgram Theorem does not apply directly . . . 11
4.3 Lax–Milgram Theorem applied to Linearized Newton subproblems . . . 11
4.4 Stability . . . 13
4.5 Empirical verification programme . . . 15

5 Results **16**
5.1 Initial conditions: proteins initial seeding and white and gray matter. . . . 16
5.2 Axonal directions. . . . 17
5.3 Sensitivity to the coefficients. . . . 18
5.4 Diffusion progression over 50 years. . . . 19
5.5 Scalability . . . 20

6 Build and Run Instructions **22**
6.1 Compilation procedure used in our experiments . . . 22
6.2 Input file structure . . . 23
6.3 CMake configuration . . . 23
6.4 Remarks on portability . . . 23

7 Conclusion **24**

References **24**

1 Introduction

Neurodegenerative diseases, such as Alzheimer's and Parkinson's, can be mathematically described using a model inspired by the mechanism of *prion diseases*. Prions, proteinaceous infectious particles, are unconventional infectious agents that consist of misfolded proteins that act as corruptive seeds to initiate a chain-reaction of misfolding, growth, and spreading. Prion diseases have some unique kinetic features: Disease progression is inevitable after initial seeding; the duration of the incubation period depends on both the initial seeding concentration and the rate at which prions amplify; and disease progression is characterized by a long, clinically silent incubation period during which prions grow and spread, followed by a brief and invariably fatal clinical disease. Interestingly, these characteristics are similar to the progression of a wide variety of age-related neurodegenerative disorders including Alzheimer's disease, Parkinson's disease, and amyotrophic lateral sclerosis.

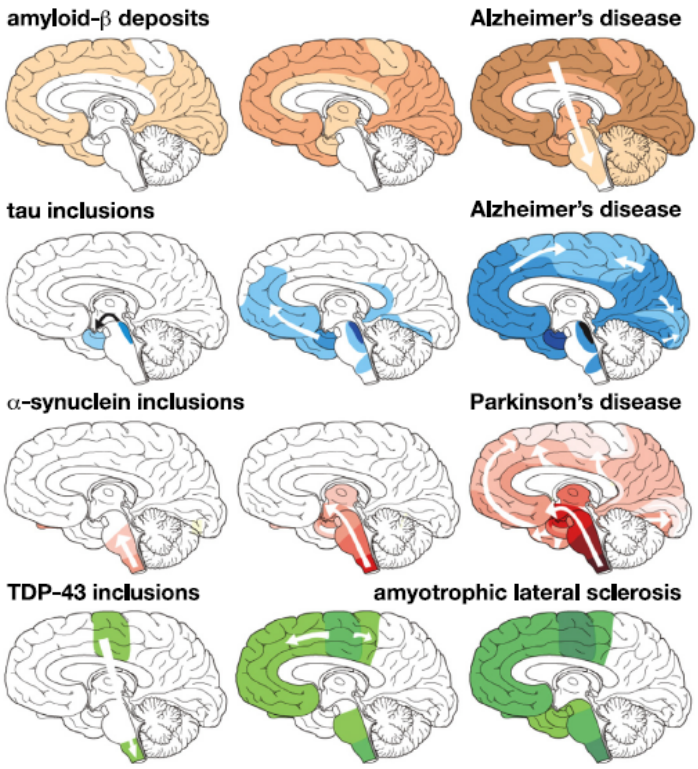


Figure 1: Three-stage progression of pathological protein spreading: Amyloid-β and tau in Alzheimer's disease, α-synuclein in Parkinson's disease, and TDP-43 in amyotrophic lateral sclerosis. Arrows indicate the hypothesized propagation pathways in brain tissue. Adapted from [1].

Interestingly, the post-mortem analysis of hundreds of human brains reveals a stereotypic progression of repeatable and predictable patterns: Amyloid-β deposits in Alzheimer's disease shown in orange in the first row first appear in the neocortex from where they spread into the allocortex and the subcortical regions; tau inclusions in Alzheimer's disease shown in blue in the second row occur first in the locus coeruleus and transentorhinal layer and then spread into the amygdala and interconnected neocortical brain regions; α-synuclein inclusions in Parkinson's disease shown in red in the third row first occur in the dorsal motor nucleus and in the anterior olfactory nucleus from where they expand in related areas evolving the brain stem and the cortex; TAR DNA-binding protein-43, TDP-43, inclusions in amyotrophic lateral sclerosis shown in green in the fourth row develop first in the agranular motor cortex, brainstem motor nuclei of cranial nerves, and spinal cord motorneurons, and then spread into the prefrontal neocortex, the brainstem, the postcentral neocortex and the striatum.

The prion paradigm could hold the key to design physics-based models that explain the stereotypic features of neurodegenerative disorders. A consistent model must explain why the disease progresses inevitably after inoculation, why the early stages of disease progress so slowly, and why a single infectious particle will spread and ultimately cause disease unless it is rapidly degraded.

2 Prion Disease Model

2.1 Kinetic Model of Prion Spreading

The following kinetic model, adapted from [1], describes the prion-like spreading of proteins in neurodegenerative diseases. We consider the simplest kinetic representation with two protein configurations: the natural healthy state (p) and a misfolded one ($\tilde{p}$). In this model, misfolded proteins recruit healthy proteins at a rate $k_{11'}$, healthy proteins bind to misfolded proteins and adopt their misfolded conformation at a rate $k_{1'2'}$, and the resulting aggregates fragment into misfolded infectious seeds at a rate $k_{2'2}$:

$$p + \tilde{p} \xrightarrow{k_{11'}} p\tilde{p} \quad p\tilde{p} \xrightarrow{k_{1'2'}} \tilde{p}\tilde{p} \quad \tilde{p}\tilde{p} \xrightarrow{k_{2'2}} \tilde{p} + \tilde{p}. \tag{1}$$

The temporal evolution of the concentrations of healthy and misfolded proteins is then described by:

$$\frac{\partial p}{\partial t} = \nabla \cdot (D_p \nabla p) + k_0 - k_1 p - k_{12} p\tilde{p}, \tag{2}$$

$$\frac{\partial \tilde{p}}{\partial t} = \nabla \cdot (D_{\tilde{p}} \nabla \tilde{p}) - \tilde{k}_1 \tilde{p} + k_{12} p\tilde{p}, \tag{3}$$

where:

- D_p and $D_{\tilde{p}}$ are the diffusion coefficients of the healthy and misfolded proteins;
- k_0 is the production rate of healthy proteins;
- k_1 and $\tilde{k}_1$ are the clearance rates of healthy and misfolded proteins, respectively;
- k_{12} is the rate at which the misfolded protein $\tilde{p}$ induces the misfolding of the healthy protein p.

2.2 From the Balance Equation to the Fisher-Kolmogorov Equation

To study the system dynamics under pathological conditions, we initially assume a *healthy* state, where the concentration of healthy proteins is much greater than that of misfolded proteins, i.e., $p \gg \tilde{p}$, which implies that $dp/dt \approx 0$ and $\nabla \cdot (D_p \nabla p) \approx 0$. From (2), we get that the concentration of p is:

$$p = \frac{k_0}{k_1 + k_{12}\tilde{p}}. \tag{4}$$

We now approximate the healthy protein concentration p using a Taylor series evaluated at $k_{12}/k_1\ \tilde{p} = 0$, to obtain:

$$p = \frac{k_0}{k_1}(1 - \tilde{p}\frac{k_{12}}{k_1}). \tag{5}$$

Substituting this expression into (3), we obtain a new equation for the evolution of $\tilde{p}$. Introducing the dimensionless variable

$$c = \frac{\tilde{p}}{\tilde{p}_{\max}}, \tag{6}$$

where $\tilde{p}_{\max}$ is the maximum concentration of misfolded proteins, defined as

$$\tilde{p}_{\max} = \frac{k_1^2}{k_0 k_{12}^2}\left(\frac{k_{12}k_0}{k_1} - \tilde{k}_1\right). \tag{7}$$

This results in the following partial differential equation, known in mathematical biology as the Fisher–Kolmogorov equation, for the misfolded protein concentration c:

$$\frac{\partial c}{\partial t} = \nabla \cdot (D \nabla c) + \alpha c(1-c), \quad \text{with} \quad \alpha = \frac{k_{12} k_0}{k_1} - \tilde{k}_1. \tag{8}$$

The diffusion term $D = D_{\tilde{p}}$ represents the diffusion of misfolded proteins in the brain and can be:

- **Isotropic**, if the diffusion is uniform in all directions;
- **Anisotropic**, if the diffusion follows specific brain structures, such as neuronal connections.

The Fisher–Kolmogorov equation has two steady state solutions, an unstable steady state at $c = 0$ and a stable steady state at $c = 1$. This implies that once misfolded protein is present anywhere in the brain, $c > 0$, the concentration will always be repelled from the benign state, $c = 0$, and attracted to the misfolded state, $c = 1$.

We can model the seeding of misfolded protein via non-homogeneous initial conditions, $c_0 > 0$, in specific brain regions. Once seeded, misfolded protein can spread across the brain by two distinct mechanisms, extracellular diffusion and axonal transport.

Here we associate extracellular diffusion d^{ext} with the isotropic diffusion of misfolded protein through the extracellular space and axonal transport d^{axn} with the anisotropic diffusion of misfolded protein along the local axonal direction n:

$$D = d^{ext} I + d^{axn} \mathbf{n} \otimes \mathbf{n}. \tag{9}$$

Prions are known to be conveyed with high specificity along established anatomical pathways and it is generally assumed that axonal transport is faster than extracellular diffusion, $d^{axn} \geq d^{ext}$.

3 Computational Model

3.1 Weak Formulation

We start with Fisher–Kolmogorov equation for the misfolded prion concentration $c(x,t) \in [0,1]$ over a spatial domain $\Omega \subset \mathbb{R}^3$ and time interval $(0,T]$:

$$\frac{\partial c}{\partial t} = \nabla \cdot (D\nabla c) + \alpha c(1-c), \tag{10}$$

with homogeneous Neumann boundary conditions

$$D\nabla c \cdot n = 0 \quad \text{on } \partial\Omega, \tag{11}$$

and an initial condition:

$$c(x,0) = c_0(x), \quad \text{for } x \in \Omega. \tag{12}$$

Let $V := H^1(\Omega)$ be the Sobolev space of square-integrable functions with square-integrable weak derivatives. The weak formulation of Equation (10) reads: find $c(x,t) \in V$ such that for all test functions $v \in V$ and $t \in (0,T]$:

$$\int_\Omega \frac{\partial c}{\partial t}\, v\, dx = \int_\Omega \nabla \cdot (D\nabla c)\, v\, dx + \int_\Omega \alpha c(1-c)\, v\, dx, \tag{13}$$

By applying Gauss-Green theorem and the Neumann boundary condition (11), we get the weak formulation:

$$\int_\Omega \frac{\partial c}{\partial t}\, v\, dx + \int_\Omega D\nabla c \cdot \nabla v\, dx = \int_\Omega \alpha c(1-c)\, v\, dx. \tag{14}$$

3.2 Residual Definition

Then we define the residual functional:

$$\mathcal{R}(c)(v) := \int_\Omega \frac{\partial c}{\partial t}\, v\, dx + \int_\Omega D\nabla c \cdot \nabla v\, dx - \int_\Omega \alpha c(1-c)\, v\, dx = 0 \quad \forall v \in V. \tag{15}$$

3.3 Spatial Discretization via Finite Elements

Let $\mathcal{T}_h$ be a conforming triangulation of Ω, and let $V_h \subset V$ be a finite-dimensional subspace consisting of continuous piecewise polynomial functions of degree r. The discrete problem reads: find $c_h(x,t) \in V_h$ such that

$$\int_\Omega \frac{\partial c_h}{\partial t}\, v_h\, dx + \int_\Omega D\nabla c_h \cdot \nabla v_h\, dx = \int_\Omega \alpha c_h(1-c_h)\, v_h\, dx, \quad \forall v_h \in V_h. \tag{16}$$

3.4 Time Discretization theta method

Let $\{t^n\}_{n=0}^N$ be a uniform partition of the time interval $[0,T]$ with step size $\Delta t = T/N$. We denote $c_h^n \approx c_h(t^n)$. Various time-stepping schemes are possible. We consider:

θ-method: The θ-method is a one-parameter family of time integration schemes, where $0 \le \theta \le 1$:

$$\begin{aligned} \int_\Omega \frac{c_h^{n+1} - c_h^n}{\Delta t}\, v_h\, dx + \int_\Omega D\nabla \left(\theta c_h^{n+1} + (1-\theta) c_h^n\right) \cdot \nabla v_h\, dx = \\ \int_\Omega \alpha \left(\theta c_h^{n+1}(1-c_h^{n+1}) + (1-\theta) c_h^n (1-c_h^n)\right) v_h\, dx. \end{aligned} \tag{17}$$

To solve the nonlinear problem arising at each time step from the θ-method discretization, we apply Newton's method. We define the time-discrete residual functional $\mathcal{R}(c_h^{n+1}) \in V_h$ as follows:

$$\mathcal{R}(c_h^{n+1})(v_h) := \int_\Omega \frac{c_h^{n+1} - c_h^n}{\Delta t} v_h \, dx + \int_\Omega D\nabla \left(\theta c_h^{n+1} + (1-\theta) c_h^n\right) \cdot \nabla v_h \, dx \\ - \int_\Omega \alpha \left(\theta c_h^{n+1}(1 - c_h^{n+1}) + (1-\theta) c_h^n (1 - c_h^n)\right) v_h \, dx. \tag{18}$$

We now compute the Fréchet derivative $a(c_h^{n+1})(\delta_h, v_h)$ of the residual in the direction of a perturbation $\delta_h \in V_h$. By definition, the Fréchet derivative satisfies:

$$\lim_{\|\delta_h\|_{V_h} \to 0} \frac{|\mathcal{R}(c_h^{n+1} + \delta_h)(v_h) - \mathcal{R}(c_h^{n+1})(v_h) - a(c_h^{n+1})(\delta_h, v_h)|}{\|\delta_h\|_{V_h}} = 0. \tag{19}$$

Step 1: Compute $\mathcal{R}(c_h^{n+1} + \delta_h)$

$$\mathcal{R}(c_h^{n+1} + \delta_h)(v_h) = \int_\Omega \frac{c_h^{n+1} + \delta_h - c_h^n}{\Delta t} v_h \, dx \\ + \int_\Omega D\nabla \left(\theta (c_h^{n+1} + \delta_h) + (1-\theta) c_h^n\right) \cdot \nabla v_h \, dx \\ - \int_\Omega \alpha \left(\theta (c_h^{n+1} + \delta_h)(1 - c_h^{n+1} - \delta_h) + (1-\theta) c_h^n (1 - c_h^n)\right) v_h \, dx. \tag{20}$$

We now expand the nonlinear term:

$$(c_h^{n+1} + \delta_h)(1 - c_h^{n+1} - \delta_h) = c_h^{n+1}(1 - c_h^{n+1}) + \delta_h(1 - 2c_h^{n+1}) - (\delta_h)^2.$$

Step 2: Compute the difference $\mathcal{R}(c_h^{n+1} + \delta_h) - \mathcal{R}(c_h^{n+1})$

$$\begin{aligned} \mathcal{R}(c_h^{n+1} + \delta_h)(v_h) - \mathcal{R}(c_h^{n+1})(v_h) &= \int_\Omega \frac{\delta_h}{\Delta t} v_h \, dx \\ &\quad + \int_\Omega D\theta \nabla \delta_h \cdot \nabla v_h \, dx \\ &\quad - \int_\Omega \alpha\theta \left[\delta_h(1 - 2c_h^{n+1}) - (\delta_h)^2\right] v_h \, dx \\ &= \int_\Omega \frac{\delta_h}{\Delta t} v_h \, dx + \int_\Omega D\theta \nabla \delta_h \cdot \nabla v_h \, dx \\ &\quad - \int_\Omega \alpha\theta (1 - 2c_h^{n+1}) \delta_h \, v_h \, dx + \int_\Omega \alpha\theta (\delta_h)^2 \, v_h \, dx. \end{aligned} \tag{21}$$

Step 3: Identify the linear part $a(c_h^{n+1})(\delta_h, v_h)$ By comparing with the linearization structure, the Fréchet derivative is given by the terms linear in δ_h:

$$a(c_h^{n+1})(\delta_h, v_h) := \int_\Omega \frac{\delta_h}{\Delta t} v_h \, dx + \int_\Omega D\theta \nabla \delta_h \cdot \nabla v_h \, dx - \int_\Omega \alpha\theta (1 - 2c_h^{n+1}) \delta_h \, v_h \, dx. \tag{22}$$

Newton iteration. At each iteration step k, Newton's method seeks a correction $\delta_h \in V_h$ such that

$$a\left(c_h^{n+1,(k)}\right)(\delta_h, v_h) = -\mathcal{R}\left(c_h^{n+1,(k)}\right)(v_h) \qquad \forall v_h \in V_h. \tag{23}$$

The new iterate is then updated as

$$c_h^{n+1,(k+1)} = c_h^{n+1,(k)} + \delta_h. \tag{24}$$

The process is repeated until convergence, measured by the norm of the residual, or until the maximum number of iterations.

3.5 Residual vector and Jacobian matrix

Let $\{\varphi_i\}_{i=1}^{N_h}$ denote a basis of the finite element space V_h that spans V_h (here $N_h = \dim V_h$). Every function $v_h \in V_h$ can be written uniquely as a linear combination of the basis functions:

$$v_h(x) = \sum_{j=1}^{N_h} v_j \, \varphi_j(x). \tag{25}$$

In particular, for the unknown at time t^{n+1} and for a Newton correction we set

$$c_h^{n+1}(x) = \sum_{j=1}^{N_h} c_j^{n+1} \, \varphi_j(x), \tag{26}$$

$$\delta_h^{(k)}(x) = \sum_{j=1}^{N_h} \delta_j^{(k)} \, \varphi_j(x). \tag{27}$$

We next introduce the standard finite element matrices and nonlinear vectors that enter the time-discrete residual and its derivative.

Finite element matrices and nonlinear vector.

- Mass matrix $M \in \mathbb{R}^{N_h \times N_h}$:

$$M_{ij} := \int_\Omega \varphi_j(x) \, \varphi_i(x) \, dx. \tag{28}$$

- stiffness matrix $K \in \mathbb{R}^{N_h \times N_h}$:

$$K_{ij} := \int_\Omega D(x) \, \nabla\varphi_j(x) \cdot \nabla\varphi_i(x) \, dx. \tag{29}$$

- Vector $F(c^{n+1}) \in \mathbb{R}^{N_h}$ with components

$$F_i(c^{n+1}) := \int_\Omega \alpha \, c_h^{n+1}(x) \big(1 - c_h^{n+1}(x)\big) \, \varphi_i(x) \, dx. \tag{30}$$

We can now rewrite the discrete residual vector $R(c^{n+1}) \in \mathbb{R}^{N_h}$ as

$$\begin{aligned} R_i(c^{n+1}) := & \int_\Omega \frac{c_h^{n+1} - c_h^n}{\Delta t} \, \varphi_i \, dx + \int_\Omega D \, \nabla\big(\theta c_h^{n+1} + (1-\theta) c_h^n\big) \cdot \nabla\varphi_i \, dx \\ & - \int_\Omega \alpha\big(\theta c_h^{n+1}(1 - c_h^{n+1}) + (1-\theta) c_h^n (1 - c_h^n)\big) \varphi_i \, dx. \end{aligned} \tag{31}$$

In compact matrix/vector notation this reads

$$R(c^{n+1}) \;=\; \frac{1}{\Delta t} \, M\big(c^{n+1} - c^n\big) \;+\; \theta \, K \, c^{n+1} \;+\; (1-\theta) \, K \, c^n \;-\; \theta \, F(c^{n+1}) \;-\; (1-\theta) \, F(c^n), \tag{32}$$

where $F(c)$ is the vector with components $F_i(c)$ defined above and c^n is the known coefficient vector at time t^n.

Moreover by taking $\delta_h = \varphi_j$ and $v_h = \varphi_i$ we obtain the Jacobian matrix, that is:

$$J_{ij}\big(c^{n+1}\big) = \frac{1}{\Delta t} \, M_{ij} \;+\; \theta \, K_{ij} \;-\; \theta \int_\Omega \alpha \, \big(1 - 2 \, c_h^{n+1}(x)\big) \, \varphi_j(x) \, \varphi_i(x) \, dx. \tag{33}$$

Equivalently, introducing the weighted mass matrix

$$W_{ij}\big(c^{n+1}\big) := \int_\Omega \alpha \, \big(1 - 2 \, c_h^{n+1}(x)\big) \, \varphi_j(x) \, \varphi_i(x) \, dx, \tag{34}$$

we can write

$$J\big(c^{n+1}\big) \;=\; \frac{1}{\Delta t} \, M \;+\; \theta \, K \;-\; \theta \, W\big(c^{n+1}\big) \, . \tag{35}$$

3.6 Implementation Details and Software Stack

The numerical solver was implemented in `C++` using the finite element library `deal.II`, together with `MPI` for distributed-memory parallelism and `Trilinos` for sparse linear algebra and preconditioning.

This software stack provides a scalable framework to translate the variational formulation derived in the previous sections into an efficient implementation on realistic three-dimensional meshes.

A primary motivation for using `deal.II` is its native support for distributed-memory finite element computations. In particular, the code employs `parallel::fullydistributed::Triangulation<3>`, so that each MPI rank stores only locally owned cells together with a ghost layer required for inter-process communication. This approach enables domain decomposition, distributed degree-of-freedom ownership, and parallel output without the need to implement low-level parallel infrastructure manually.

The solver is written in terms of the polynomial degree r specified in the input file. The corresponding finite element space is defined using `FE_SimplexP<dim>(r)`, allowing the method to naturally extend from linear ($P1$) to higher-order simplicial elements. Gaussian quadrature on simplices is introduced through `QGaussSimplex<dim>(r+1)`, consistently with the polynomial degree of the approximation.

The connection between the mathematical formulation and the implementation appears most clearly in the assembly stage. Using the `FEValues` class, shape functions, gradients, quadrature points, and Jacobian weights (`JxW`) are evaluated at Gauss points, allowing the mass, diffusion, and reaction terms to be assembled exactly in the form derived in the weak formulation and in the Newton linearization.

The resulting distributed sparse system is assembled using the sparsity pattern generated by `DoFTools::make_sparsity_pattern` and stored in `TrilinosWrappers::SparseMatrix`. The linear systems arising at each Newton iteration are solved iteratively. In the final version of the solver, we adopted `MINRES` combined with an `AMG` preconditioner from `Trilinos`, which provided a good compromise between efficiency and robustness in the numerical experiments. In section 4.5, a more detailed discussion of the solver and preconditioner selection is presented.

Overall, `deal.II` acts as a reliable software layer that preserves the variational structure of the finite element formulation while enabling scalable distributed-memory computations on complex three-dimensional geometries.

3.7 Implementation of the Diffusion Tensor

A key modeling ingredient of the Fisher–Kolmogorov equation is the diffusion tensor D, which governs the preferential spreading of misfolded proteins through brain tissue. At the continuous level, we consider the anisotropic form

$$D = d^{ext} I + d^{axn}\, \mathbf{n} \otimes \mathbf{n}, \tag{36}$$

where d^{ext} represents extracellular diffusion, d^{axn} models transport along axonal pathways, and $\mathbf{n}$ is a unit vector describing the local preferred propagation direction.

In the implementation, the tensor D is represented by the class `FunctionD`, which evaluates the diffusion tensor pointwise at quadrature points. This is important because the tensor may vary in space and may also depend on the tissue type. For each quadrature point x_q, the code computes a local tensor $D(x_q)$, which is then used in the bilinear form through the quantity

$$(D(x_q)\nabla\varphi_j(x_q)) \cdot \nabla\varphi_i(x_q).$$

As a consequence, anisotropy is taken into account directly during assembly, rather than through a post-processing correction.

The implementation supports four different choices for the axonal field:

- **isotropic field:** $D = d^{ext} I$;
- **radial field:** the unit direction $\mathbf{n}$ points radially away from a prescribed center;
- **circular field:** the preferred direction is tangent to circular trajectories around the center;

- **axon-based field:** the direction is defined piecewise, combining a circular structure in an inner region with a radial structure outside it.

These four cases allow us to compare purely isotropic spreading with increasingly structured anisotropic propagation patterns.

The code also supports an optional distinction between white and gray matter.

When `matter_type=1`, cells are assigned a material identifier during mesh initialization. White matter cells use the full anisotropic tensor (36), whereas gray matter cells use an isotropic tensor

$$D = d^{ext} I.$$

This reflects the modeling assumption that long-range axonal transport primarily occurs in white matter, while diffusion in gray matter is predominantly extracellular. In addition, the reaction coefficient is reduced in gray matter through

$$\alpha_{\text{gray}} = \frac{\alpha}{2},$$

which introduces a simple but effective tissue-dependent heterogeneity in the model.

From a numerical point of view, this design is convenient because the tensor is evaluated locally at quadrature points and immediately inserted into the element matrix. Therefore, the passage from the mathematical model to the code is very direct: the diffusion operator remains exactly the one derived in the weak formulation, but with a coefficient field that may be isotropic or anisotropic, homogeneous or heterogeneous, depending on the simulation parameters.

4 Well–posedness and Stability

The Fisher-Kolmogorov equation is a nonlinear equation, so classical linear tools (such as Lax-Milgram theorem) cannot be applied to the problem. In this section we explain this limitation, indicate what can still be proved and then describe the empirical verification programme we adopt to assess stability and well-posedness of the numerical scheme.

4.1 Standing assumptions.

We assume that $\Omega \subset \mathbb{R}^d$ is a Lipschitz domain, $D \in L^\infty(\Omega; \mathbb{R}^{d\times d})$ is symmetric and uniformly elliptic, i.e.,

$$\exists\, \underline{D} > 0 \ \text{ such that } \ \xi^\top D(x)\,\xi \ \geq \ \underline{D}\,|\xi|^2, \quad \forall\, \xi \in \mathbb{R}^d, \ \text{for a.e. } x \in \Omega,$$

$\alpha \geq 0$, $\Delta t > 0$, $\theta \in [0,1]$, and $c_h^{(k)} \in V_h \subset H^1(\Omega)$, with $0 \leq c_h^{(k)} \leq 1$ a.e. in Ω. In particular, the L^∞ norm of the diffusion tensor D is defined as

$$\|D\|_{L^\infty(\Omega)} = \operatorname*{ess\,sup}_{x\in\Omega} \|D(x)\|_{\mathrm{op}}.$$

Since $D(x)$ is symmetric and uniformly elliptic, its operator norm coincides with its largest eigenvalue, hence

$$\|D\|_{L^\infty(\Omega)} = \sup_{x\in\Omega} \lambda_{\max}(D(x)).$$

4.2 Why Lax–Milgram Theorem does not apply directly

The weak formulation contains the nonlinear reaction term

$$\alpha c(1-c)\,,$$

which depends nonlinearly on the unknown solution c. Lax–Milgram requires a linear form; hence it cannot be applied directly to the nonlinear problem. Similarly, the classical "consistency+stability $\Rightarrow$ convergence" argument is valid for linear problems but does not provide a proof for our nonlinear scheme.

4.3 Lax–Milgram Theorem applied to Linearized Newton subproblems

At each time step the nonlinear equation (17) is solved by Newton's method. Each Newton correction requires the solution of a *linear* variational problem of the following form:

We seek $\delta_h \in V_h \subset H^1(\Omega)$ such that

$$\int_\Omega \frac{\delta_h}{\Delta t}\, v_h\, dx + \int_\Omega D\theta \nabla \delta_h \cdot \nabla v_h\, dx - \int_\Omega \alpha\theta(1-2c_h^{(k)})\delta_h\, v_h\, dx = -\mathcal{R}^{(k)}(v_h) \qquad \forall\, v_h \in V_h \subset H^1(\Omega). \tag{37}$$

where $c_h^{(k)}$ is the current Newton iterate and $\mathcal{R}^{(k)}$ is the nonlinear residual. Although the PDE is parabolic, after θ-time discretization each step reduces to a purely spatial (elliptic) problem. The Newton linearization is thus a linear elliptic variational problem.

Bilinear form. The associated bilinear form is

$$B(w,v) = \int_\Omega \frac{w}{\Delta t}\, v\, dx + \int_\Omega D\theta \nabla w \cdot \nabla v\, dx - \int_\Omega \alpha\theta(1-2c_h^{(k)}) w\, v\, dx. \tag{38}$$

Testing with $v = w$ gives

$$\begin{aligned} B(w,w) &= \frac{1}{\Delta t}\int_\Omega w^2\, dx + \int_\Omega D\theta \nabla w \cdot \nabla w\, dx - \int_\Omega \alpha\theta(1-2c_h^{(k)}) w^2\, dx \\ &= \frac{1}{\Delta t}\|w\|^2_{L^2(\Omega)} + \int_\Omega D\theta \nabla w \cdot \nabla w\, dx - \int_\Omega \alpha\theta(1-2c_h^{(k)}) w^2\, dx. \end{aligned} \tag{39}$$

Immediate consequences. By uniform ellipticity, $\int_\Omega D\nabla w\cdot\nabla w\,dx \ge \underline{D}\,\|\nabla w\|^2_{L^2(\Omega)}$.

Moreover, $0 \le c_h^{(k)} \le 1$ a.e. implies $\|1-2c_h^{(k)}\|_{L^\infty(\Omega)} \le 1$.

Strong coercivity under a time–step condition. Testing with $v = w$ and using uniform ellipticity of D and $|1-2c_h^{(k)}| \le 1$ a.e. we obtain

$$\begin{aligned} B(w,w) &= \frac{1}{\Delta t}\|w\|^2_{L^2(\Omega)} + \theta\int_\Omega D\nabla w\cdot\nabla w\,dx - \theta\int_\Omega \alpha(1-2c_h^{(k)})\,w^2\,dx \\ &\ge \Big(\tfrac{1}{\Delta t} - \theta\alpha\Big)\|w\|^2_{L^2(\Omega)} \;+\; \theta\,\underline{D}\,\|\nabla w\|^2_{L^2(\Omega)}. \end{aligned} \tag{40}$$

Hence, if

$$\boxed{\tfrac{1}{\Delta t} > \theta\,\alpha},$$

both coefficients are strictly positive and, setting $m := \min\{\frac{1}{\Delta t} - \theta\alpha,\ \theta\,\underline{D}\} > 0$,

$$B(w,w) \;\ge\; m\Big(\|w\|^2_{L^2(\Omega)} + \|\nabla w\|^2_{L^2(\Omega)}\Big) \;=\; m\,\|w\|^2_{H^1(\Omega)} \qquad \forall\, w \in H^1(\Omega). \tag{41}$$

That is, B is (strongly) coercive on $H^1(\Omega)$.

Weak coercivity. $B(\cdot,\cdot)$ is not strongly coercive in general. However, we can establish global *weak coercivity*. Let

$$\lambda := \theta\alpha\|1-2c^{(k)}\|_{L^\infty(\Omega)} \le \theta\alpha. \tag{42}$$

Then

$$B(w,w) + \lambda\|w\|^2_{L^2(\Omega)} = \frac{1}{\Delta t}\|w\|^2_{L^2(\Omega)} + \theta\int_\Omega D\nabla w\cdot\nabla w\,dx. \tag{43}$$

By previous assumptions,

$$B(w,w) + \lambda\|w\|^2_{L^2(\Omega)} \;\ge\; \frac{1}{\Delta t}\|w\|^2_{L^2(\Omega)} + \theta\underline{D}\|\nabla w\|^2_{L^2(\Omega)}. \tag{44}$$

Thus, for all $w \in H^1(\Omega)$,

$$B(w,w) + \lambda\|w\|^2_{L^2(\Omega)} \;\ge\; \alpha_0\|w\|^2_{H^1(\Omega)}, \qquad \alpha_0 := \min\Big\{\tfrac{1}{\Delta t},\ \theta\underline{D}\Big\}. \tag{45}$$

This is precisely the weak coercivity inequality required.

Continuity. The bilinear form $B(\cdot,\cdot)$ is continuous on $H^1(\Omega)\times H^1(\Omega)$. Indeed,

$$|B(w,v)| \le \Big(\tfrac{1}{\Delta t} + \theta\|\alpha(1-2c_h^{(k)})\|_{L^\infty(\Omega)}\Big)\|w\|_{L^2(\Omega)}\|v\|_{L^2(\Omega)} + \theta\|D\|_{L^\infty(\Omega)}\|\nabla w\|_{L^2(\Omega)}\|\nabla v\|_{L^2(\Omega)}. \tag{46}$$

By the Cauchy–Schwarz inequality and the definition of the H^1 norm, this gives

$$|B(w,v)| \;\le\; C\|w\|_{H^1(\Omega)}\|v\|_{H^1(\Omega)}, \qquad C := \tfrac{1}{\Delta t} + \theta\alpha \;+\theta\|D\|_{L^\infty(\Omega)}. \tag{47}$$

Hence B is continuous with continuity constant C.

Continuity of the right-hand side. Let $F^{(k)} : H^1(\Omega) \to \mathbb{R}$ be the linear functional

$$\begin{aligned} F^{(k)}(v_h) &:= -\mathcal{R}\Big(c_h^{n+1,(k)}\Big)\,(v_h) \\ &= -\int_\Omega \frac{c_h^{n+1,(k)} - c_h^n}{\Delta t}\,v_h\,dx \;-\; \int_\Omega D\,\nabla\big(\theta c_h^{n+1,(k)} + (1-\theta)c_h^n\big)\cdot\nabla v_h\,dx \\ &\quad + \int_\Omega \alpha\,\Big[\theta\,c_h^{n+1,(k)}\big(1-c_h^{n+1,(k)}\big) + (1-\theta)c_h^n(1-c_h^n)\Big]\,v_h\,dx. \end{aligned} \tag{48}$$

Using the Cauchy–Schwarz inequality and $\|D\|_{L^\infty(\Omega)} = \operatorname{ess\,sup}_{x\in\Omega} \|D(x)\|_{\mathrm{op}}$, for all $v_h \in H^1(\Omega)$ we get

$$\begin{aligned} |F^{(k)}(v_h)| \le \frac{1}{\Delta t} \|c_h^{n+1,(k)} - c_h^n\|_{L^2(\Omega)} \|v_h\|_{L^2(\Omega)} \\ + \|D\|_{L^\infty(\Omega)} \Big(\theta \|\nabla c_h^{n+1,(k)}\|_{L^2(\Omega)} + (1-\theta) \|\nabla c_h^n\|_{L^2(\Omega)}\Big) \|\nabla v_h\|_{L^2(\Omega)} \\ + \alpha \Big(\theta \|c_h^{n+1,(k)}(1-c_h^{n+1,(k)})\|_{L^2(\Omega)} + (1-\theta) \|c_h^n(1-c_h^n)\|_{L^2(\Omega)}\Big) \|v_h\|_{L^2(\Omega)}. \end{aligned} \tag{49}$$

Set

$$\begin{aligned} a &:= \frac{1}{\Delta t} \|c_h^{n+1,(k)} - c_h^n\|_{L^2(\Omega)} + \alpha \Big(\theta \|c_h^{n+1,(k)}(1-c_h^{n+1,(k)})\|_{L^2(\Omega)} + (1-\theta) \|c_h^n(1-c_h^n)\|_{L^2(\Omega)}\Big), \\ b &:= \|D\|_{L^\infty(\Omega)} \Big(\theta \|\nabla c_h^{n+1,(k)}\|_{L^2(\Omega)} + (1-\theta) \|\nabla c_h^n\|_{L^2(\Omega)}\Big). \end{aligned}$$

Then, viewing $\big(\|v_h\|_{L^2}, \|\nabla v_h\|_{L^2}\big)$ as a vector in $\mathbb{R}^2$ and using Cauchy–Schwarz,

$$|F^{(k)}(v_h)| \;\le\; a\,\|v_h\|_{L^2(\Omega)} + b\,\|\nabla v_h\|_{L^2(\Omega)} \;\le\; \sqrt{a^2+b^2}\,\sqrt{\|v_h\|^2_{L^2(\Omega)} + \|\nabla v_h\|^2_{L^2(\Omega)}} \;=\; C_F^{(k)}\,\|v_h\|_{H^1(\Omega)}, \tag{50}$$

with

$$C_F^{(k)} := \sqrt{a^2+b^2}\,.$$

Well-posedness via Lax–Milgram. The bilinear form B is bilinear, continuous and, if $\frac{1}{\Delta t} > \theta\,\alpha$, then B is coercive on $H^1(\Omega) \times H^1(\Omega)$. Moreover, the right-hand side functional $F^{(k)}$ is linear and continuous on $H^1(\Omega)$. Hence, by the classical Lax–Milgram theorem, assuming $\frac{1}{\Delta t} > \theta\,\alpha$, there exists a unique $\delta_h \in V_h \subset H^1(\Omega)$ such that

$$B(\delta_h, v_h) = F^{(k)}(v_h) \qquad \forall v_h \in H^1(\Omega),$$

and $\|\delta_h\|_{H^1(\Omega)} \le C\,\|F^{(k)}\|_{H^1(\Omega)}$.

4.4 Stability

Energy stability (Gronwall inequality). We derive an L^2 energy bound without using Poincaré, suitable for pure Neumann boundary conditions. To obtain an *a priori* L^2 bound we test the PDE (10) with $v_h = c_h$, i.e. we multiply by the solution itself and integrate over Ω. This gives

$$\int_\Omega \frac{\partial c_h}{\partial t}\, c_h\, dx + \int_\Omega D\nabla c_h \cdot \nabla c_h\, dx = \alpha \int_\Omega c_h^2(1-c_h)\, dx. \tag{51}$$

The first term is

$$\int_\Omega \frac{\partial c_h}{\partial t}\, c_h\, dx = \tfrac{1}{2}\tfrac{d}{dt}\|c_h\|^2_{L^2(\Omega)}. \tag{52}$$

For the reaction term we note that, exploiting $0 \le c_h \le 1$ a.e.,

$$\int_\Omega c_h^2(1-c_h)\, dx = \|c_h\|^2_{L^2(\Omega)} - \|c_h\|^3_{L^3(\Omega)} \le \|c_h\|^2_{L^2(\Omega)}, \tag{53}$$

since the cubic term is nonnegative. Hence

$$\frac{1}{2}\frac{d}{dt}\|c_h\|^2_{L^2} + \int_\Omega D|\nabla c_h|^2\, dx \le \alpha\|c_h\|^2_{L^2}. \tag{54}$$

which is by previous assumptions at 4.1,

$$\frac{d}{dt}\|c_h\|^2_{L^2} + 2\underline{D}\|\nabla c_h\|^2_{L^2(\Omega)} \le 2\alpha\|c_h\|^2_{L^2}. \tag{55}$$

set $y(t) := \|c_h(t)\|^2_{L^2(\Omega)}$. Then

$$\frac{1}{2}\, y'(t) + \underline{D}\, \|\nabla c_h(t)\|^2_{L^2(\Omega)} \;\le\; \alpha\, y(t). \tag{56}$$

Discarding the nonnegative gradient term yields $\frac{1}{2}y'(t) \le \alpha\, y(t)$, and by Gronwall's inequality

$$y(t) \;\le\; e^{2\alpha t}\, y(0) \qquad \Longleftrightarrow \qquad \|c_h(t)\|^2_{L^2(\Omega)} \;\le\; e^{2\alpha t}\, \|c_h(0)\|^2_{L^2(\Omega)}, \quad t \in [0,T]. \tag{57}$$

Integrating (56) over $(0,T)$ and using that $-\frac{1}{2}\, y(T) \le 0$ gives

$$\underline{D} \int_0^T \|\nabla c_h\|^2_{L^2(\Omega)}\, dt \;\le\; \alpha \int_0^T y(t)\, dt \;+\; \tfrac{1}{2}\, y(0). \tag{58}$$

Using (57) we obtain

$$\alpha \int_0^T y(t)\, dt \;\le\; \alpha\, y(0) \int_0^T e^{2\alpha t}\, dt = \frac{e^{2\alpha T} - 1}{2}\, y(0) \;\le\; \frac{e^{2\alpha T}}{2}\, y(0), \tag{59}$$

and therefore

$$\underline{D} \int_0^T \|\nabla c_h\|^2_{L^2(\Omega)}\, dt \;\le\; \frac{e^{2\alpha T}}{2}\, \|c_h(0)\|^2_{L^2(\Omega)}. \tag{60}$$

In particular,

$$c_h \in L^\infty(0,T; L^2(\Omega)) \cap L^2(0,T; H^1(\Omega)),$$

with constants growing at most like $\frac{e^{2\alpha T}}{2}$.

4.5 Empirical verification programme

Given these limitations, we adopt a systematic empirical strategy to assess stability and convergence.

Stability sweeps. Vary systematically the parameters $(\Delta t, \theta, \alpha, d_{\text{axn}}, d_{\text{ext}})$ and record:

- Newton convergence (success/failure, iterations per step),
- L^2 norm history $\|c_h(\cdot, t)\|_{L^2}$,

This allows to map stability regions empirically: When fixing $\theta = 1$ (backward Euler), we observe a trade-off between Δt and the growth rate α: larger values of α introduce sharper temporal variations, which require smaller time steps to ensure convergence. Conversely, fixing Δt to the baseline value, we notice a similar trade-off happening for θ and α, smaller values of θ make the system more similar to a forward scheme and therefore easier to solve but less robust. The less robust scheme is then more susceptible to the divergence induced by α. An additional observation is that the diffusion coefficient D never induces numerical divergence, which is expected since higher diffusion makes the solution smoother and therefore easier to find.

Solver and Preconditioner Given the form of the diffusion coefficient D, the resulting system matrices are symmetric, allowing us to employ the efficient iterative solvers available in `deal.II`. The final solver choice was guided by empirical testing of different combinations of Krylov methods and preconditioners. Specifically, we considered the GMRES, MINRES, and Conjugate Gradient (CG) solvers, together with SSOR, ILU, and AMG preconditioners. Benchmark runs indicated that both MINRES and CG combined with an ILU preconditioner yielded the fastest convergence. However, ILU does not guarantee robustness, as it breaks the solvers' assumption of symmetric matrices. For this reason, our final choice was MINRES with AMG, which provided a good balance between robustness and performance in the numerical experiments.

The empirical testing results on a simulation with the baseline values and $T = 4$ are presented in Table 1

Pair tested	Newton Iteration	Time step	Total time
GMRES + SSOR	7.30	21.92	175.38
GMRES + ILU	2.56	11.20	89.60
GMRES + AMG	1.51	12.51	100.06
MINRES + SSOR	5.70	15.55	124.40
MINRES + ILU	2.17	10.24	81.90
MINRES + AMG	1.79	10.77	86.22
CG + SSOR	5.32	15.33	122.68
CG + ILU	1.90	10.32	82.61
CG + AMG	1.86	11.20	89.62

Table 1: Average elapsed time (s) for different preconditioner-solver pairs.

5 Results

We now present numerical experiments illustrating the behavior of the prion propagation model under different conditions. The simulations were carried out on a realistic brain geometry discretized by finite elements, and the results are visualized as concentration maps of the misfolded proteins. The baseline parameters used for the various simulations were: Total time = 20 years, $\Delta t = 0.5$ years, axonal diffusion $d_{\text{axn}} = 10\ mm^2/\text{year}$, external diffusion $d_{\text{ext}} = 5\ mm^2/\text{year}$ and $\alpha = 0.25$. In addition, the initial conditions of the TDP-43 protein, the distinction between white and gray matter, and the axon-based axonal directions were considered. All the parameters were then modified independently to study the effect of each one. These experiments serve two purposes: to demonstrate the ability of the computational scheme to capture nonlinear front propagation, and to highlight the role of diffusion anisotropy, reaction rates, and initial conditions in shaping the progression of pathology.

5.1 Initial conditions: proteins initial seeding and white and gray matter.

Firstly, we show the four possible initial conditions corresponding to the four proteins under study. Each condition is defined by setting a concentration value greater than zero within geometrically prescribed regions of the domain, chosen to approximate realistic patterns of early pathological inclusions.

Figure 2: amyloid-β deposits

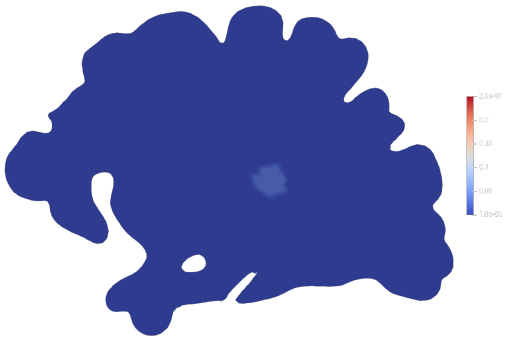


Figure 3: Tau inclusions

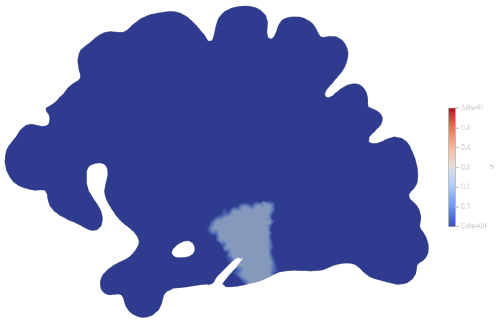


Figure 4: α-synuclein inclusions

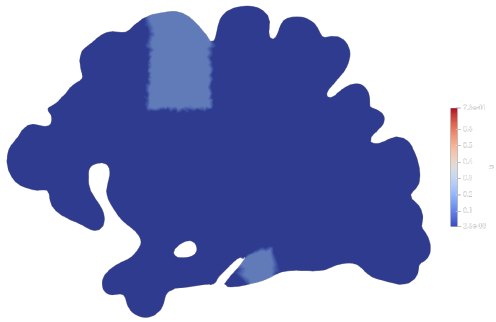


Figure 5: TDP-43 inclusions

Another important factor to be considered for the simulation is the type of brain matter present in different regions of the brain. White matter forms the internal parts of the brain, where both extracellular and axonal diffusions can act; Gray matter is instead found near the surface and only the extracellular diffusion takes action.

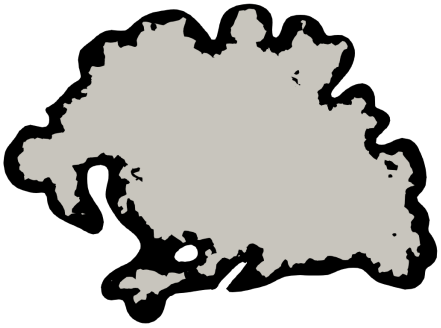


Figure 6: Subdivision of the brain into white and gray (in black) matter

5.2 Axonal directions.

The first relevant set of results shows how different implementations of the direction of the axons influence the distribution of the proteins. In the first scenario, we consider purely isotropic diffusion. Over time, the concentration spreads uniformly, producing nearly spherical fronts that expand steadily. This confirms the classical Fisher–Kolmogorov wave-like behavior, with the front speed determined by the diffusion strength D and the growth rate α. In the other three scenarios we introduce anisotropy in the diffusion tensor to reflect axonal fiber alignment. In those cases, spreading occurs preferentially along axonal directions giving rise to elongated fronts that respect the underlying structural connectivity.

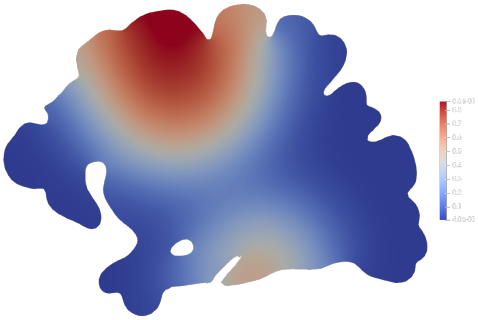


Figure 7: Isotropic axons

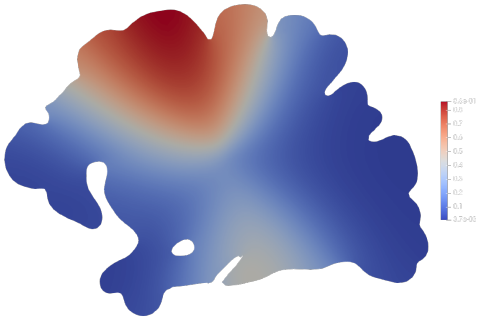


Figure 8: Radial axonal directions

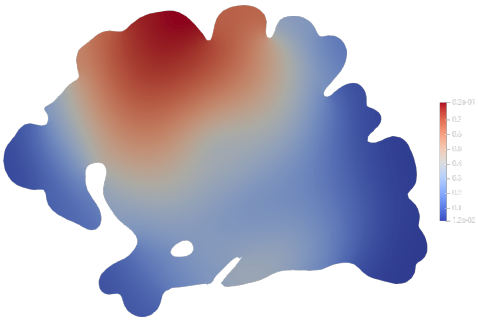


Figure 9: Circular axonal directions

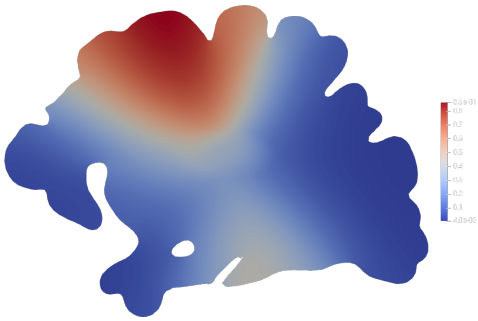


Figure 10: Axon-based axonal directions

5.3 Sensitivity to the coefficients.

The next step in the project was to study the different behaviors obtained by modifying the different parameters of the simulation. Starting from the diffusion coefficient D, we notice that, as expected, a low value of the diffusion parameter ends in a very high concentration of misfolded proteins around the initial seeding. By increasing the diffusion parameter, the proteins spread faster, obtaining a less concentrated and more uniform distribution throughout the entire brain.

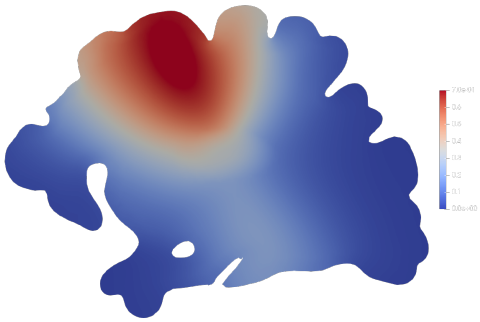


Figure 11: Baseline values for D

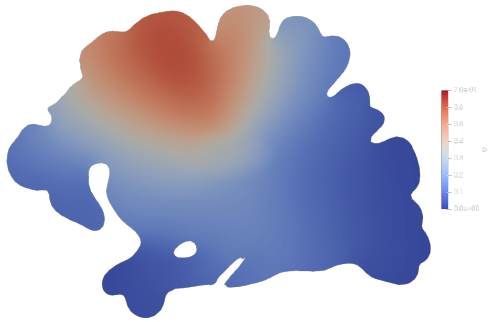


Figure 12: Values of D doubled

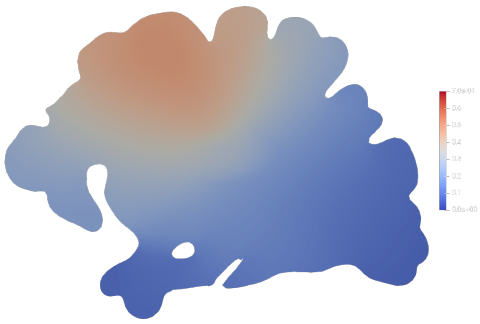


Figure 13: Values of D quadrupled

Figure 14: Values of D octupled

The other parameter to be studied is the growth rate α. At low values, after 20 years, we find a critical concentration of misfolded proteins only around the initial seeding areas. By increasing α, the conversion rate towards a misfolded state is higher and the critical density is reached much faster in most areas of the brain.

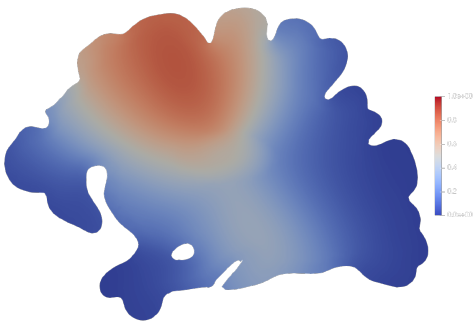


Figure 15: $\alpha = 0.3$

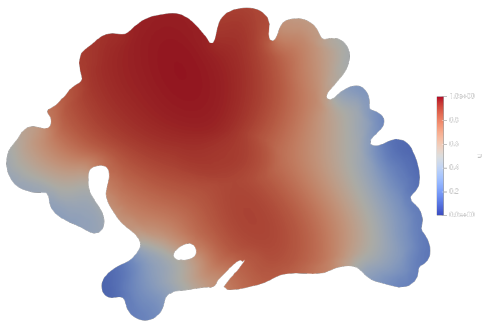


Figure 16: $\alpha = 0.45$

Figure 17: $\alpha = 0.6$

Figure 18: $\alpha = 1.2$

5.4 Diffusion progression over 50 years.

In the last set of results we show the progression of the diffusion of α-synuclein inclusions over a longer period of time. As expected the simulation converges towards the stable configuration at concentration equal to 1 over the entire brain; the initial seedings given by the other proteins follow the same pattern.

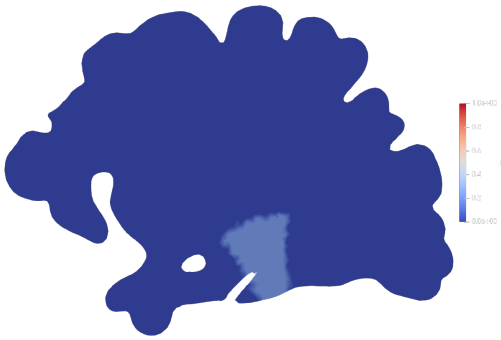


Figure 19: Year 0

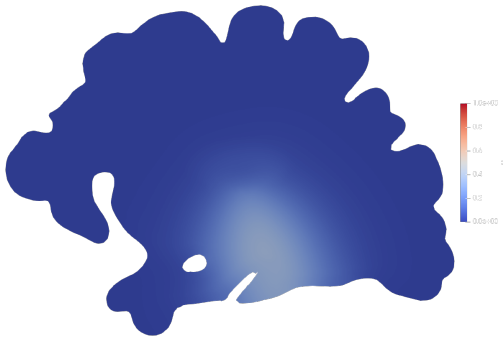


Figure 20: Year 10

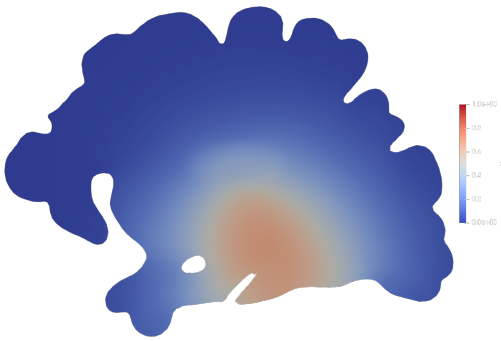


Figure 21: Year 20

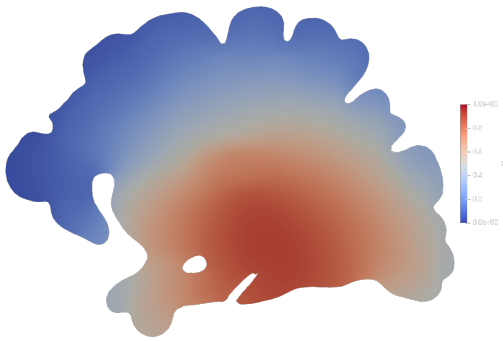


Figure 22: Year 30

Figure 23: Year 40

Figure 24: Year 50

Comparing the diffusion of the various proteins, taking the center point of the brain as benchmark, we can notice that the moment in time and speed at which the critical concentration is reached depends heavily on the initial conditions: Amyloid-β deposits are seeded already occupying a wide area and reach criticality first. At the opposite end, Tau inclusions start from a very small, low concentration initial seeding and reach criticality much later.

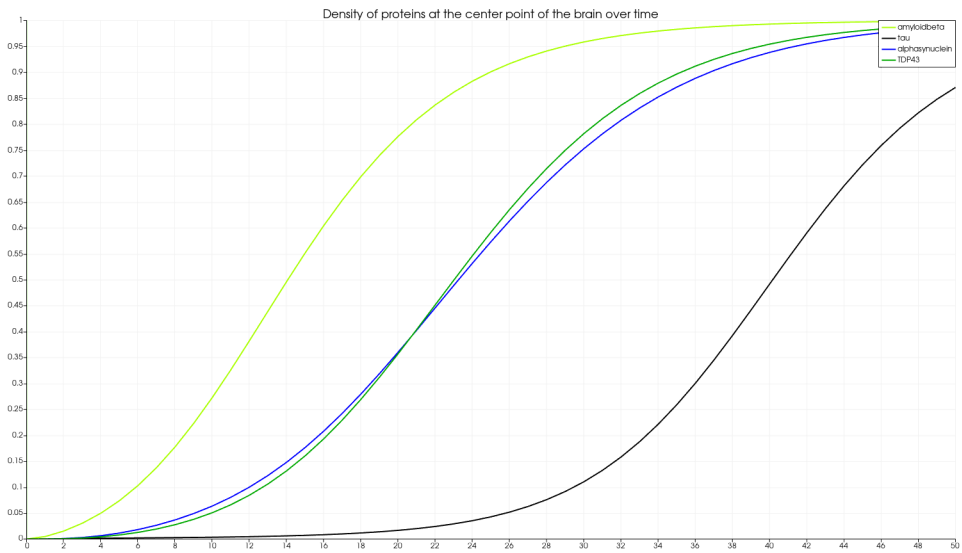


Figure 25: Density of each protein at the center point over time

5.5 Scalability

To assess the parallel performance of our solver, a strong scalability study was carried out on the **MeluXina** EuroHPC supercomputer. The tests were executed on the `gpu` partition that we had available in the project contract, but the runs were CPU-only (no GPUs requested). Each compute node in this partition is equipped with two AMD EPYC Rome 7452 processors (32 cores each, for a total of 64 physical cores, 128 threads with SMT) running at 2.35 GHz, and 512 GB of RAM.

We fixed the spatial discretization (mesh and problem size) and increased the number of MPI ranks from 1 up to 128. For each run we collected:

- the **Execution Time**, i.e. the elapsed wall-clock time to complete the simulation;
- the **TotalCPU time**, reported by Slurm as the sum of CPU times over all allocated processes;
- the **memory consumption**, monitored as
  - **AveRSS**: average Resident Set Size across processes;
  - **MaxRSS**: maximum Resident Set Size across processes.

The results are reported in Table 2. Execution Time decreases nearly ideally up to 32 processes, while at 64 processes it still shows good strong scaling. The run with 128 processes, however, was distributed over two nodes (`-N 2, --ntasks-per-node=64`), and performance degraded significantly due to the higher cost of inter-node communication and synchronization. This behavior is expected for domain-decomposed finite element solvers, where ghost-layer exchanges and sparse matrix assembly incur non-negligible communication overhead once crossing the node boundary.

Table 2: Strong scalability results on MeluXina (1 node up to 64 ranks, 2 nodes for 128 ranks).

Processes	**Execution time [s]**	**TotalCPU [h:mm:ss]**	**AveRSS [MB]**	**MaxRSS [MB]**
1	2422.912	00:40:23	1386.785	3036.592
2	1336.381	00:54:16	2250.618	3504.412
4	751.122	01:00:53	3594.989	5425.112
8	436.718	01:12:50	4458.447	5691.932
16	237.379	01:20:50	6304.562	7554.640
32	122.979	01:32:03	10738.000	13276.768
64	86.459	02:21:33	20697.175	25657.416
128	340.400	06:28:47	41313.585	46238.548

Figure 26 shows the strong scaling of the Execution Time. In the log–log plot, the ideal strong scaling is represented by a straight line of slope -1. Our results closely follow the ideal reference up to 32 processes, while at 64 processes scalability remains favorable but with some communication overhead. At 128 processes the curve bends upwards, reflecting the degradation caused by inter-node communication.

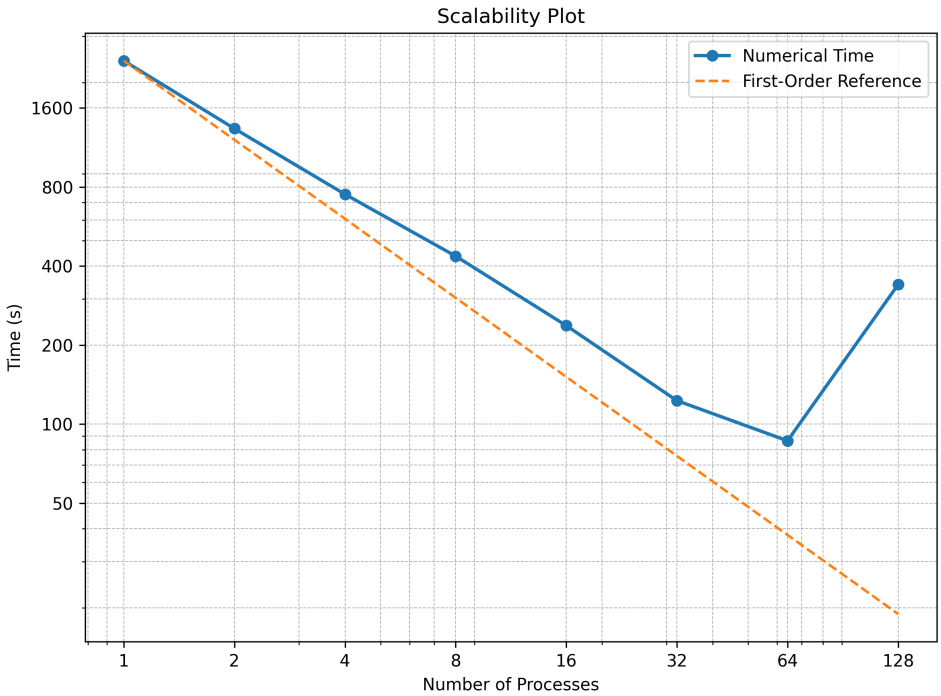


Figure 26: Execution Time vs number of processes (strong scaling). The dashed line represents ideal scaling of slope -1 in log–log scale.

Figure 27 reports the memory usage. Memory consumption increases with the number of processes, partly due to the duplication of local data structures (e.g. ghost cells, communication buffers). The growth remains moderate up to 32 processes, but becomes more significant beyond 64 processes.

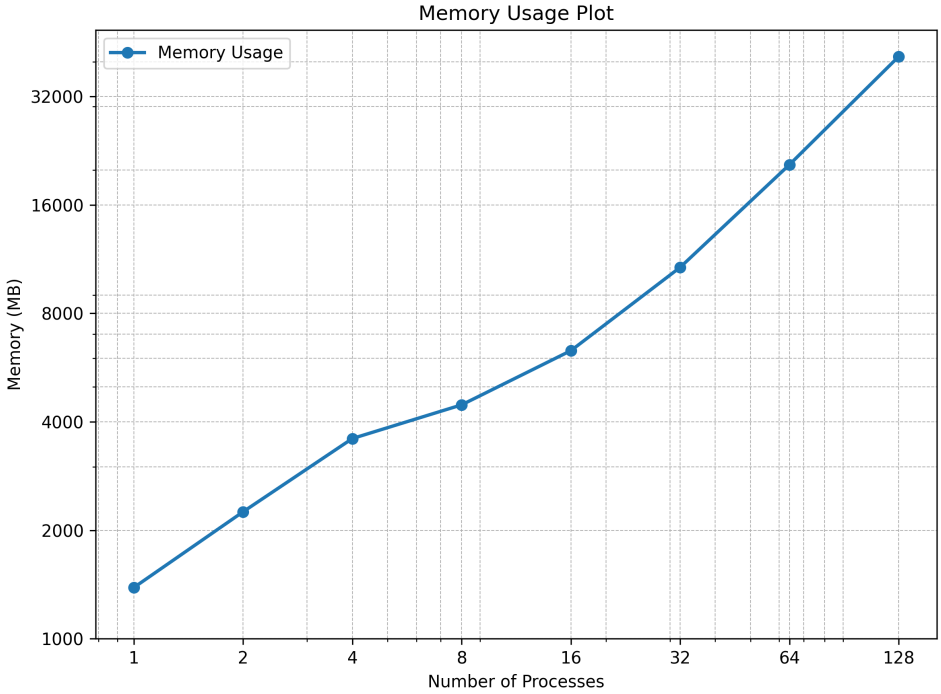


Figure 27: Memory usage vs number of processes (AveRSS).

6 Build and Run Instructions

The project is built with `CMake` and requires a `C++17` compiler, `MPI`, `deal.II`, and `Trilinos`. In the development workflow, the code was compiled inside a Docker container providing a controlled software environment with the required libraries already installed.

6.1 Compilation procedure used in our experiments

The following commands reproduce the build procedure used for the experiments reported in this work:

```
docker start hpc-env
docker exec -it hpc-env /bin/bash

cd /home/jellyfish/shared-folder/fk-solver

module load gcc-glibc dealii

rm -rf build
mkdir build
cd build

cmake -Ddeal.II_DIR=/u/sw/toolchains/gcc-glibc/11.2.0/pkgs/dealii/9.5.1/lib/cmake/deal.II ..
make -j4
```

The executable is then launched with `mpirun`. A typical execution command is:

```
mpirun -n 4 ./FisherKolmogorov ../src/parameters.csv
```

6.2 Input file structure

The executable reads a CSV file containing one row per simulation. For example:

```
mesh_file_name,degree,T,deltat,theta,matter_type,protein_type,
axonal_field,d_axn,d_ext,alpha,output_dir
../mesh/mesh-cube-20.msh,1,2.0,0.5,1.0,0,4,4,10.0,5.0,0.25,output
```

In this example, the code runs a single simulation on the tetrahedral mesh `mesh-cube-20.msh` with linear finite elements ($r = 1$), final time $T = 2$, time step $\Delta t = 0.5$, fully implicit time discretization ($\theta = 1$), TDP-43 initial condition (`protein_type=4`), axon-based anisotropy (`axonal_field=4`), and output written to the folder `output/`. For this configuration, the initialization stage reported:

- 48000 mesh elements,
- polynomial degree 1,
- 4 degrees of freedom per cell,
- 6 quadrature points per cell,
- 9261 global degrees of freedom.

6.3 CMake configuration

The project is configured through the following `CMakeLists.txt` file:

```
cmake_minimum_required(VERSION 3.16)
project(FisherKolmogorov LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(deal.II 9.4 CONFIG REQUIRED)

deal_ii_initialize_cached_variables()

add_executable(FisherKolmogorov
  src/FisherKolmogorov.cpp
  src/DiffusionNonLinear.cpp
)

deal_ii_setup_target(FisherKolmogorov)
```

Although the generic `CMakeLists.txt` only requires `deal.II` version 9.4 or higher, the MeluXina-specific configuration additionally checks for `MPI`, `Trilinos`, and `p4est` support, since these components are needed by the parallel distributed triangulation used in the project.

6.4 Remarks on portability

The same project structure was used both locally and on the MeluXina supercomputer. On MeluXina, the build remained `CMake`-based, with only minor adaptations to the configuration: in particular, the cluster-specific setup checked explicitly that the available `deal.II` installation had been compiled with `MPI`, `Trilinos`, and `p4est`, which are required by the distributed-memory implementation based on `parallel::fullydistributed::Triangulation`.

For the scalability campaign, compilation and execution were automated through dedicated Slurm scripts, which loaded the appropriate modules, rebuilt the executable in release mode, and launched the MPI job with explicit core binding. These scripts are included in the repository under `scripts/`, together with the MeluXina-specific `CMake` configuration.

7 Conclusion

In this work, we presented a finite element solver for the Fisher–Kolmogorov equation applied to the spreading of misfolded proteins in neurodegenerative diseases. Starting from the prion-inspired biological model, we derived the weak formulation, introduced the θ-scheme for time discretization, and solved the nonlinear problem at each time step through Newton's method.

From the computational viewpoint, the project shows that the model can be implemented effectively on realistic 3D meshes while retaining both flexibility and scalability.

The use of anisotropic diffusion tensors, heterogeneous white/gray matter coefficients, and protein-dependent initial conditions allows the solver to reproduce qualitatively different spreading patterns.

Moreover, the strong-scaling experiments on MeluXina confirm that the parallel implementation is effective up to one node, with the expected degradation only when inter-node communication becomes dominant.

Overall, the results indicate that the Fisher–Kolmogorov framework provides a meaningful mathematical and computational model for studying pathological protein propagation, while the `deal.II/MPI/Trilinos` implementation offers a solid basis for further developments, such as higher-order discretizations, more realistic anatomical datasets, and patient-specific simulations.

References

[1] J. Weickenmeier, M. Jucker, A. Goriely, and E. Kuhl. A physics-based model explains the prion-like features of neurodegeneration in Alzheimer's disease, Parkinson's disease, and amyotrophic lateral sclerosis. *Journal of the Mechanics and Physics of Solids*, 124:264–281, 2019.
https://www.sciencedirect.com/science/article/pii/S0022509618307063

[2] M. Corti, F. Bonizzoni, L. Dedè, A. M. Quarteroni, and P. F. Antonietti. Discontinuous Galerkin Methods for Fisher–Kolmogorov Equation with Application to α-Synuclein Spreading in Parkinson's Disease. *arXiv preprint*, arXiv:2302.07126, 2023.
https://arxiv.org/pdf/2302.07126

[3] LuxProvide. *Meluxina User Documentation*. System Overview and Handling Jobs.
https://docs.lxp.lu/

[4] W. Bangerth, R. Hartmann, G. Kanschat et al. *The deal.II Library, Version 9.4: A modern C++ software library for the finite element method.*
https://dealii.org/9.4.1/index.html

[5] M. A. Heroux et al. *Trilinos User Guide*. Sandia National Laboratories, 2020.
https://trilinos.github.io/pdfs/TrilinosUserGuide.pdf